

How do you run CS223 labs on a Mac without Vivado?

19 September 2026 · M2, 8 GB, macOS 15 [PDF](#) [the five project files](#) section 1 is the one-line install, 2 to 4 the setup by hand, 5 to 8 your lab and the limits

Apple Silicon Mac (M1 or later), Digilent Basys3, free tools. Tested on an M2 with 8 GB of RAM and macOS 15 on 19 September 2026. Intel Macs are untested.

There are two ways in. Both end at the same place: `.sv` in, `.bit` on the board, about 4 seconds a build.

	One line	By hand
time	~4 min, one command	~20 min, 7 steps
you get	the <code>dewfpga</code> command	a Makefile you own and understand
read	section 1	sections 2 to 4

Sections 5 to 8 apply to both: your own lab, VS Code, the course file that breaks, what this chain cannot do.

No Mac or no board with you? The [Basys3 simulator](#) and the [testbench runner](#) run the same `.sv` and `.xdc` in the browser, nothing installed.

If a step fails. Read the last lines in the terminal. Every error I hit is filed under its exact text on the [errors page](#), with the fix. If yours is not there, send me the last ten lines and the section number on LinkedIn.

Contents

- [1. How do you install it in one line?](#)
 - [2. What do you need before you start?](#)
 - [3. How do you build the chain by hand?](#)
 - [4. How do you build with the Makefile?](#)
 - [5. How do you use it for your own lab?](#)
 - [6. How do you set up VS Code?](#)
 - [7. Which course file breaks?](#)
 - [8. What can it not do?](#)
 - [9. Which versions did I use?](#)
-

1. How do you install it in one line?

You need Xcode Command Line Tools and Homebrew (section 2 has both, two commands). Then:

```
curl -fsSL https://nosey-dewdrop.github.io/dewfpga/install | bash    # ~4 min, 1.4 GB, once
```

It puts the CLI in ~/.dewfpga, the tools in ~/fpga, and links dewfpga into Homebrew's bin. Re-run the same line to update. Nothing touches your system Python.

Make the example project and flash it:

```
dewfpga new blink
cd blink
dewfpga sim      # simulation: TB: basic checks passed
dewfpga bit      # .sv -> .bit
dewfpga flash    # to the board: LED 15 follows switch 0, LED 0 blinks
```

What dewfpga bit prints, in full:

```
xdc ok: 33 ports, all mapped.
pnr ok: 73 LUT, 27 FF, 278.71 MHz (PASS at 100.00 MHz)    (full log: blink.log)
blink.bit 2.2 MB
```

The three lines are the pin check, place and route, and the bitstream. The full nextpnr log goes to blink.log so you can read it when something is off.

That is the whole tool. In any folder, every .sv and .v is synthesized, so submodules can live in their own files. The top module is the .sv with a matching .xdc; if that is ambiguous, name it: dewfpga flash <top>. sim needs <top>_tb.sv.

dewfpga install	install the toolchain (safe to re-run)
dewfpga check	is every piece in place
dewfpga sim bit flash clean [top]	work on the .sv files in the current folder
dewfpga new <dir>	blink example with a VS Code task (%↑B = flash)

If you took this path, skip to [section 5](#).

2. What do you need before you start?

Open the terminal: Cmd Space, type Terminal, Enter. Every command below goes there. Paste one, press Enter, wait for the prompt to come back.

Apple's command line tools give you git and clang.

```
xcode-select --install
```

Homebrew installs developer tools by name.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

It asks for your Mac password; the letters do not show while you type. At the end it prints a few lines under "Next steps". Paste those too, or the next command answers command not found. Check with `brew --version`.

A code editor. VS Code, from code.visualstudio.com. Not TextEdit, which saves rich text the tools cannot read.

3. How do you build the chain by hand?

Vivado does five jobs. One free tool does each one here:

```
your_design.sv
|-> Icarus Verilog    -> simulation
|-> Yosys             -> synthesis
|-> nextpnr-xilinx    -> place and route
|-> fasm2frames        -> configuration frames
|-> xc7frames2bit      -> your_design.bit
|-> openFPGALoader    -> the board, over USB
```

Three come from Homebrew. Two you compile. One database you generate. Everything lands in `~/fpga`, 1.76 GB in total. Vivado asks for 50 to 100 GB.

3.1 Homebrew packages

```
brew install yosys openfpgaloader icarus-verilog \
  cmake ninja boost eigen pkg-config libomp python
```

The first three are tools from the diagram. The rest are what the next two builds need. A few hundred MB.

Check:

```
yosys -V          # Yosys 0.69+post
openFPGALoader --Version # v1.1.1
iverilog -V | head -1    # Icarus Verilog version 13.0 (stable)
```

Newer numbers are fine.

3.2 Python venv

Two steps of the chain are Python scripts. Homebrew's Python refuses a plain `pip install` (externally-managed-environment), so they get a private one.

```
mkdir -p ~/fpga
python3 -m venv ~/fpga/venv
~/fpga/venv/bin/pip install fasm pyyaml textx simplejson intervaltree
```

3.3 nextpnr-xilinx

Place and route. Homebrew has no macOS build, so you compile it. About 1.5 minutes.

```
cd ~/fpga
git clone --recursive --depth 1 https://github.com/openXC7/nextpnr-xilinx.git
cd nextpnr-xilinx
cmake -B build -G Ninja \
  -DARCH=xilinx -DCMAKE_BUILD_TYPE=Release \
  -DUSE_OPENMP=OFF -DBUILD_GUI=OFF -DBUILD_PYTHON=OFF -DBUILD_TESTS=OFF
ninja -C build -j3
```

`-DUSE_OPENMP=OFF` is required: Apple's compiler has no OpenMP (fopenmp). `-j3` fits 8 GB of RAM; raise it on 16 GB or more.

Check:

```
~/fpga/nextpnr-xilinx/build/nextpnr-xilinx --version
# "nextpnr-xilinx" -- Next Generation Place and Route (Version 0.9.6)
```

3.4 Chip database

nextpnr needs a description of the XC7A35T. It is not in the download; you generate it once (no-chipdb). About 40 seconds.

```
mkdir -p ~/fpga/chipdb
cd ~/fpga/nextpnr-xilinx
~/fpga/venv/bin/python xilinx/python/bbaexport.py \
  --xray xilinx/external/prjxray-db/artix7 \
  --metadata xilinx/external/nextpnr-xilinx-meta/artix7 \
  --device xc7a35tcpg236-1 \
  --constids xilinx/constids.inc \
  --bba ~/fpga/chipdb/xc7a35t.bba
build/bbasm -l ~/fpga/chipdb/xc7a35t.bba ~/fpga/chipdb/xc7a35t.bin
```

Another 7-series board: change `--device`. Untested.

3.5 prjxray

Turns nextpnr's text output into the binary .bit. One Python script, one compiled tool. About 2 minutes.

```
cd ~/fpga
git clone --recursive --depth 1 https://github.com/f4pga/prjxray.git
cd prjxray
~/fpga/venv/bin/pip install -e .
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_POLICY_VERSION_MINIMUM=3.5
cmake --build build --target xc7frames2bit -j3
```

--recursive is required ([yaml-cpp-not-built](#)). -DCMAKE_POLICY_VERSION_MINIMUM=3.5 is required ([cmake-policy-version-minimum](#)). prjxray's own README says to install Vivado 2017.2. Ignore it; that is for mapping a new chip, and ours is mapped.

Check:

```
ls ~/fpga/prjxray/build/tools/xc7frames2bit
```

Prints the path back: the toolchain is installed.

4. How do you build with the Makefile?

Your project is a folder with five files. Download them, do not type them:

```
mkdir -p ~/cs223/blink && cd ~/cs223/blink
curl -L -o blink.zip https://nosey-dewdrop.github.io/dewfpga/blink.zip && unzip blink.zip
```

file	what it is
blink.sv	the design: LED 0 blinks once a second while switch 0 is on, LED 15 follows switch 0
blink_tb.sv	the testbench make sim runs
blink.xdc	the pin file. Digilent's Basys3 master XDC with the 33 pin lines for clk, sw and led uncommented. Works as-is in Vivado too
check_xdc.py	compares the design's ports with the XDC before place and route, because nextpnr's own error names the wrong port
Makefile	runs the five tools in order. --seed 1 makes every build identical

If you copied the Makefile from somewhere else and make says missing separator, the tabs became spaces: [missing-separator](#).

Plug the board in over USB, switch it on, then:

```
make check      # one line per tool; none should say MISSING
make sim        # TB: basic checks passed
make bit        # .sv -> .bit, ~4 seconds
make flash      # to the board
```

`make bit` prints three lines: `xdc ok`, `pnr ok` with the LUT and flip-flop count and the clock check, and the `.bit` size. The full nextpnr log is in `blink.log`. `make sim` also writes `blink.vcd`; the testbench runner draws it, since no waveform viewer is installed here.

A good `make flash` ends with:

```
Load SRAM: [=====] 100.00%
Done
ir: 1 isc_done 1 isc_ena 0 init 1 done 1
```

`done 1` means the chip accepted the design. Switch 0 on: LED 15 lights, LED 0 blinks. The design lives in SRAM, so a power cycle clears it and you flash again.

5. How do you use it for your own lab?

With dewfpga: put your `.sv` files and the `.xdc` in one folder and run `dewfpga flash`. The top module is the `.sv` whose name matches the `.xdc`.

With the Makefile: copy `Makefile` and `check_xdc.py` into the lab folder and change the first three lines.

```
TOP := traffic_light
SRCS := traffic_light.sv debounce.sv
XDC := lab4.xdc
```

Either way, the port names in your top module must match the names in the pin file: Digilent's file says `clk`, `sw[0]`, `led[0]`, `seg[0]`, `an[0]`, `btnC`. If your module says `clock`, change the name inside `[get_ports ...]` on that line. The `xdc` check names every mismatch before place and route starts. Simulation looks for `<top>_tb.sv`.

6. How do you set up VS Code?

```
code --install-extension mshr-h.veriloghdl
```

If `code` is not found: in VS Code, `Cmd Shift P`, "Shell Command: Install 'code' command in PATH".

`dewfpga new` and `blink.zip` both include `.vscode/tasks.json`, so `Cmd Shift B` builds and flashes the open design. For your own folder, copy that file from [templates/.vscode/tasks.json](#).

7. Which course file breaks?

One. SevSeg_4digit.sv from the course has a port line that Vivado and Icarus accept and Yosys does not (port-neither-input-nor-output).

```
output [6:0]seg, logic dp,      // dp has no direction
```

Write it as two lines and it synthesizes.

```
output [6:0] seg,  
output logic dp,
```

Everything else from CS223 went through with zero errors. I ran real student code from old repos: an FSM with typedef enum, a 16x8 RAM, a debouncer, the seven-segment driver. All four together: 136 LUTs, 48 flip-flops, timing passes at 154.70 MHz against the 100 MHz clock, bitstream in 4.5 seconds. Language features that passed: parameter, generate, struct packed, \$clog2, unique case, packed arrays, interface with modport. File names that differ from the module name, Turkish characters, a BOM and spaces in the path all passed too.

8. What can it not do?

This covers the part of Vivado that CS223 uses, and no more.

- **Nothing from Vivado's IP Catalog.** No Block Design, MicroBlaze, AXI, Clocking Wizard, or the BRAM, VGA and UART cores. I searched 35 old student repos and 240 source files; none of them use any of it.
- **No GUI.** No waveform viewer, schematic or in-chip debugger. sim writes a .vcd; the testbench runner draws it in the browser.
- **Two parsers.** Icarus reads your code for simulation, Yosys for synthesis, and they do not support the same SystemVerilog. Code that passes sim can fail in bit; section 7 is the real example. Rewriting the line more simply has fixed it every time.
- **The timing number is an estimate** from nextpnr's model. Vivado's report is the official one. At 100 MHz with lab-sized designs the margin is large.
- **Unofficial tools.** AMD does not make or support them; prjxray worked out the bitstream format without AMD's documentation. Every lab I tried worked. If a lab is graded in Vivado, open it in Vivado once before the demo.
- **Arrays become distributed RAM** (-nobram). Fine at lab sizes. Big multipliers and DSP blocks are untested.
- **One board, one platform.** Basys3 (XC7A35T), Apple Silicon. Intel Mac, Linux and other boards are untested and the installer refuses them.
- **Not verified on hardware:** the combined four-module test above went as far as a bitstream. Loaded and confirmed on the board: blink, switches-to-LEDs, and the Lab 2

adder/subtractor.

9. Which versions did I use?

The one-line installer pins nextpnr-xilinx and prjxray to the exact commits below. `git clone` by hand brings the newest; if a step that worked for me fails for you, that is the first thing to suspect.

Yosys	0.69+post (git 143eb14f)
nextpnr-xilinx	0.9.6 (openXC7, commit 3fd7878)
prjxray	commit c9f02d8, prjxray-db 0.9.1-30-g1768fb3 (artix7)
openFPGALoader	1.1.1
Icarus Verilog	13.0 (stable)
Board	Digilent Basys3, Artix-7 XC7A35T-1CPG236C
Machine	Apple M2, 8 GB RAM, macOS 15 (Darwin 24.2.0)
